

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В POSTGRESQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Чан Тхи Лиен

Факультет прикладной информатики

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург

2025/2026

1. Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2. Программное обеспечение: СУБД PostgreSQL, SQL Shell (psql)

3. Практическое задание

- Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
- Создать триггеры для индивидуальной БД согласно варианту:
 - Вариант 2.1. 7 триггеров по ИЗ.
 - Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

4. Выполнение: Вариант 2.1

4.1. Создать хранимые процедуры

а) Для повышения цен в пригородные поезда на 20%.

```
CREATE OR REPLACE PROCEDURE "Passengers".raise_suburban_prices()
```

```
LANGUAGE plpgsql AS $$
```

```
BEGIN
```

```
UPDATE "Passengers"."Тип билета"
```

```
SET "Базовая цена" = "Базовая цена" * 1.2
```

```
WHERE "Название" ILIKE '%Пригород%';
```

```
RAISE NOTICE 'Цены на пригородные билеты успешно повышены на 20%%';
```

```
END;
```

```
$$;
```

```
TranLien=# CREATE OR REPLACE PROCEDURE "Passengers".raise_suburban_prices()
TranLien=# LANGUAGE plpgsql AS $$
TranLien$$ BEGIN
TranLien$$     UPDATE "Passengers"."Тип билета"
TranLien$$     SET "Базовая цена" = "Базовая цена" * 1.2
TranLien$$     WHERE "Название" ILIKE '%Пригород%';
TranLien$$     RAISE NOTICE 'Цены на пригородные билеты успешно повышены на 20%%';
TranLien$$ END;
TranLien$$ $$;
CREATE PROCEDURE
```

- Проверка

- Смотрим текущие цены, чтобы зафиксировать исходное состояние.

```
TranLien=# SELECT * FROM "Passengers"."Тип билета";
```

ID_типа билета	Название	Базовая цена	Правила скидок
2	Детский	1750	50%
3	Студенческий	2600	25%
4	Льготный	1200	66%
5	Люкс	9500	Нет
6	Бизнес	15000	Нет
7	Групповой	2500	От 10 чел
8	Военный	1800	Спец
9	Пенсионный	1750	50%
1	Полный	6048	Нет
10	Пригородный	1296	Акция

- Запускаем процедуру повышения цен.

```
TranLien=# CALL "Passengers".raise_suburban_prices();
NOTICE:  0aiu ia i?eai?iaiaua aeeau oniaoii iiauoaiu ia 20%
CALL
```

Появилось уведомление: Цены на пригородные билеты успешно повышены на 20%

- Снова проверяем таблицу. Цены должны вырасти на 20%.

```
TranLien=# SELECT * FROM "Passengers"."Тип билета";
```

ID_типа билета	Название	Базовая цена	Правила скидок
2	Детский	1750	50%
3	Студенческий	2600	25%
4	Льготный	1200	66%
5	Люкс	9500	Нет
6	Бизнес	15000	Нет
7	Групповой	2500	От 10 чел
8	Военный	1800	Спец
9	Пенсионный	1750	50%
1	Полный	6048	Нет
10	Пригородный	1555	Акция

Цена пригородного повышалась на 20%: 1296 —> 1555

b) Для создания нового рейса на поезд.

```
CREATE OR REPLACE PROCEDURE "Passengers".create_new_trip(
    p_id_trip integer,
    p_type_train_id integer,
    p_train_num integer,
    p_date date
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO "Passengers"."Рейс" (
        "ID_рейса", "ID_типа поезда", "Номер поезда",
```

```

        "Плановая дата", "Фактическая дата", "Статус"
    )
    VALUES (p_id_trip, p_type_train_id, p_train_num, p_date, NULL, 'Активен');
    RAISE NOTICE 'Новый рейс % успешно добавлен в расписание.', p_id_trip;
END;
$$;

```

```

TranLien=# CREATE OR REPLACE PROCEDURE "Passengers".create_new_trip(
TranLien(#      p_id_trip integer,
TranLien(#      p_type_train_id integer,
TranLien(#      p_train_num integer,
TranLien(#      p_date date
TranLien(# )
TranLien=# LANGUAGE plpgsql AS $$
TranLien$$ BEGIN
TranLien$$     INSERT INTO "Passengers"."Рейс" (
TranLien$$         "ID_рейса", "ID_типа поезда", "Номер поезда",
TranLien$$         "Плановая дата", "Фактическая дата", "Статус"
TranLien$$     )
TranLien$$     VALUES (p_id_trip, p_type_train_id, p_train_num, p_date, NULL, 'Активен');
TranLien$$     RAISE NOTICE 'Новый рейс % успешно добавлен в расписание.', p_id_trip;
TranLien$$ END;
TranLien$$ $$;
CREATE PROCEDURE

```

- Проверка
- Смотрим текущие рейсы

```

TranLien=# SELECT * FROM "Passengers"."Рейс";
ID_рейса | ID_типа поезда | Номер поезда | Плановая дата | Фактическая дата | Плановое время | Фактическое время | Статус
-----+-----+-----+-----+-----+-----+-----+-----
    100 |          1 |         701 | 2026-05-06 |                  |                |                  | Активен
    200 |          3 |         702 | 2026-05-18 |                  |                |                  | Активен
    300 |          4 |         715 | 2026-05-19 |                  |                |                  | Активен
(3 rows)

```

- Запускаем

```

TranLien=# CALL "Passengers".create_new_trip(400, 5, 777, CURRENT_DATE + 1);
NOTICE:  Iiaue ?aen 400 oniaoii aiaaaeeai a ?anienaiea.
CALL

```

- Снова проверяем таблицу

```

TranLien=# SELECT * FROM "Passengers"."Рейс";
ID_рейса | ID_типа поезда | Номер поезда | Плановая дата | Фактическая дата | Плановое время | Фактическое время | Статус
-----+-----+-----+-----+-----+-----+-----+-----
    100 |          1 |         701 | 2026-05-06 |                  |                |                  | Активен
    200 |          3 |         702 | 2026-05-18 |                  |                |                  | Активен
    300 |          4 |         715 | 2026-05-19 |                  |                |                  | Активен
    400 |          5 |         777 | 2026-05-19 |                  |                |                  | Активен

```

Рейс 400 был добавлен

- Для формирования общей выручки по продаже билетов за сутки.

```

CREATE OR REPLACE PROCEDURE "Passengers".get_daily_revenue(INOUT total_sum
numeric)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT COALESCE(SUM(tb."Базовая цена"), 0) INTO total_sum
    FROM "Passengers"."Билет" b
    JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета"
    WHERE b."Дата прибытия" = CURRENT_DATE;
END;
$$;

```

```

TranLien=# CREATE OR REPLACE PROCEDURE "Passengers".get_daily_revenue(INOUT total_sum numeric)
TranLien=# LANGUAGE plpgsql AS $$
TranLien$$ BEGIN
TranLien$$     SELECT COALESCE(SUM(tb."Базовая цена"), 0) INTO total_sum
TranLien$$     FROM "Passengers"."Билет" b
TranLien$$     JOIN "Passengers"."Тип билета" tb ON b."ID_типа билета" = tb."ID_типа билета"
TranLien$$     WHERE b."Дата прибытия" = CURRENT_DATE;
TranLien$$ END;
TranLien$$ $$;
CREATE PROCEDURE

```

- Проверка

```

TranLien=# CALL "Passengers".get_daily_revenue(0);
total_sum
-----
      5200

```

4.2. Создать 7 необходимых триггеров

Перед запуском создадим вспомогательную таблицу для хранения логов автоматического аудита:

```

CREATE TABLE IF NOT EXISTS "Passengers"."logs" (
    id serial PRIMARY KEY,
    log_text text,
    log_time timestamp DEFAULT now()
);

```

```

TranLien=# CREATE TABLE IF NOT EXISTS "Passengers"."logs" (
TranLien(#      id serial PRIMARY KEY,
TranLien(#      log_text text,
TranLien(#      log_time timestamp DEFAULT now()
TranLien(# );
CREATE TABLE

```

a) ТРИГГЕР 1: Логирование регистрации новых билетов (AFTER INSERT)

```

CREATE OR REPLACE FUNCTION "Passengers".fn_log_ticket_purchase() RETURNS
TRIGGER AS $$
BEGIN
    INSERT INTO "Passengers"."logs" (log_text)
        VALUES ('Оформлен новый билет ID: ' || NEW."ID_билета" || ' со статусом: ' ||
NEW."Статус");
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_log_ticket_ins AFTER INSERT ON "Passengers"."Билет"
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_log_ticket_purchase();

```

```

TranLien=# CREATE TABLE IF NOT EXISTS "Passengers"."logs" (
TranLien(#      id serial PRIMARY KEY,
TranLien(#      log_text text,
TranLien(#      log_time timestamp DEFAULT now()
TranLien(# );
CREATE TABLE
TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_log_ticket_purchase() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$#     INSERT INTO "Passengers"."logs" (log_text)
TranLien$#     VALUES ('Оформлен новый билет ID: ' || NEW."ID_билета" || ' со статусом: ' || NEW."Статус");
TranLien$#     RETURN NEW;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_log_ticket_ins AFTER INSERT ON "Passengers"."Билет"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_log_ticket_purchase();
CREATE TRIGGER

```

- Проверка

- Оформляем покупку нового билета (ID=801).

```

TranLien=# INSERT INTO "Passengers"."Билет" ("ID_билета", "ID_типа билета", "ID_паспорта", "ID_кассы", "Дата прибытия", "Статус")
TranLien=# VALUES (801, 1, 1, 1, CURRENT_DATE, 'Оформлен');
INSERT 0 1

```

- Проверяем, сработал ли триггер и записалась ли строка в таблицу аудита.

```

TranLien=# SELECT * FROM "Passengers"."logs" ORDER BY id DESC LIMIT 1;
 id |          log_text          |          log_time
-----+-----+-----
  1 | Оформлен новый билет ID: 801 со статусом: Оформлен | 2026-05-18 07:29:57.307963
(1 row)

```

b) ТРИГГЕР 2: Предотвращение изменения уже оплаченных билетов (BEFORE UPDATE)

```

CREATE OR REPLACE FUNCTION "Passengers".fn_protect_paid_tickets() RETURNS
TRIGGER AS $$

```

```

BEGIN

```

```

    IF OLD."Статус" = 'Оплачен' AND NEW."Статус" <> 'Оплачен' THEN

```

```

        RAISE EXCEPTION 'Запрещено изменять статус или аннулировать уже
оплаченный билет №%!', OLD."ID_билета";

```

```

    END IF;

```

```

    RETURN NEW;

```

```

END;

```

```

$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_protect_tickets BEFORE UPDATE ON "Passengers"."Билет"
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_protect_paid_tickets();

```

```

TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_protect_paid_tickets() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$#     IF OLD."Статус" = 'Оплачен' AND NEW."Статус" <> 'Оплачен' THEN
TranLien$#         RAISE EXCEPTION 'Запрещено изменять статус или аннулировать уже оплаченный билет №%!', OLD."ID_билета";
TranLien$#     END IF;
TranLien$#     RETURN NEW;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_protect_tickets BEFORE UPDATE ON "Passengers"."Билет"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_protect_paid_tickets();
CREATE TRIGGER

```

- Проверка

```

TranLien=# UPDATE "Passengers"."Билет" SET "Статус" = 'Оформлен' WHERE "ID_билета" = 501;
ERROR:  Cai?auaii eciaiyou noaooп eee aiiоee?iaaou o?a iiea?aiiue aeeao ?501!
CONTEXT:  PL/pgSQL function "Passengers".fn_protect_paid_tickets() line 4 at RAISE

```

с) ТРИГГЕР 3: Проверка корректности статуса при вводе касс (BEFORE INSERT/UPDATE)

```
CREATE OR REPLACE FUNCTION "Passengers".fn_validate_kassa_status() RETURNS
TRIGGER AS $$
```

```
BEGIN
```

```
IF NEW."Статус" NOT IN ('Да', 'Нет') THEN
```

```
RAISE EXCEPTION 'Некорректный статус кассы: %. Разрешено только "Да" или
"Нет".', NEW."Статус";
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_val_kassa BEFORE INSERT OR UPDATE ON
"Passengers"."Касса"
```

```
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_validate_kassa_status();
```

```
TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_validate_kassa_status() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$# IF NEW."Статус" NOT IN ('Да', 'Нет') THEN
TranLien$# RAISE EXCEPTION 'Некорректный статус кассы: %. Разрешено только "Да" или "Нет".', NEW."Статус";
TranLien$# END IF;
TranLien$# RETURN NEW;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_val_kassa BEFORE INSERT OR UPDATE ON "Passengers"."Касса"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_validate_kassa_status();
CREATE TRIGGER
```

- Проверка

```
TranLien=# INSERT INTO "Passengers"."Касса" ("ID_кассы", "Тип продаж", "Город", "Адрес", "Номер тел", "Статус")
TranLien=# VALUES (99, 'Офлайн', 'Москва', 'Тест', '123', 'Неизвестно');
ERROR:  Iaеi??аеоiue noаоon eаnnu: Iaесааnoii. ?ас?аоaii оiеuei "Аа" еее "Iао".
CONTEXT:  PL/pgSQL function "Passengers".fn_validate_kassa_status() line 4 at RAISE
```

Так как статус: Неизвестно —> ошибка

```
TranLien=# INSERT INTO "Passengers"."Касса" ("ID_кассы", "Тип продаж", "Город", "Адрес", "Номер тел", "Статус")
TranLien=# VALUES (99, 'Офлайн', 'Москва', 'Тест Вокзал', '777', 'Да');
INSERT 0 1
```

Статус: Да —> Добавление

d) ТРИГГЕР 4: Защита от удаления активных рейсов (BEFORE DELETE)

```
CREATE OR REPLACE FUNCTION "Passengers".fn_prevent_active_trip_del() RETURNS
TRIGGER AS $$
```

```
BEGIN
```

```
IF OLD."Статус" = 'Активен' THEN
```

```
RAISE EXCEPTION 'Нельзя удалить рейс %, так как он имеет статус Активен!',
OLD."ID_рейса";
```

```
END IF;
```

```
RETURN OLD;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_no_del_trip BEFORE DELETE ON "Passengers"."Рейс"
```

```
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_prevent_active_trip_del();
```

```

TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_prevent_active_trip_del() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$#     IF OLD."Статус" = 'Активен' THEN
TranLien$#         RAISE EXCEPTION 'Нельзя удалить рейс %, так как он имеет статус Активен!', OLD."ID_рейса";
TranLien$#     END IF;
TranLien$#     RETURN OLD;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_no_del_trip BEFORE DELETE ON "Passengers"."Рейс"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_prevent_active_trip_del();
CREATE TRIGGER

```

```

TranLien=# DELETE FROM "Passengers"."Рейс" WHERE "ID_рейса" = 100;
ERROR:  Iaeуcy oaaeeou ?aen 100, oae eae ii eiaao noaoon Aeoeaai!
CONTEXT:  PL/pgSQL function "Passengers".fn_prevent_active_trip_del() line 4 at RAISE

```

```

TranLien=# INSERT INTO "Passengers"."Рейс" ("ID_рейса", "ID_типа поезда", "Номер поезда", "Плановая дата", "Статус")
TranLien=# VALUES (201, 3, 705, CURRENT_DATE, 'Отменен');
INSERT 0 1
TranLien=# DELETE FROM "Passengers"."Рейс" WHERE "ID_рейса" = 201;
DELETE 1

```

е) ТРИГГЕР 5: Контроль вместимости вагонов при добавлении (BEFORE INSERT/UPDATE)

```

CREATE OR REPLACE FUNCTION "Passengers".fn_check_wagon_seats() RETURNS
TRIGGER AS $$

```

```

BEGIN

```

```

    IF NEW."Количество мест" <= 0 OR NEW."Количество мест" > 64 THEN

```

```

        RAISE EXCEPTION 'Ошибка! Вместимость вагона должна быть от 1 до 64 мест.

```

```

Введено: %', NEW."Количество мест";

```

```

    END IF;

```

```

    RETURN NEW;

```

```

END;

```

```

$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_wagon_seats BEFORE INSERT OR UPDATE ON
"Passengers"."Вагон"

```

```

FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_check_wagon_seats();

```

```

TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_check_wagon_seats() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$#     IF NEW."Количество мест" <= 0 OR NEW."Количество мест" > 64 THEN
TranLien$#         RAISE EXCEPTION 'Ошибка! Вместимость вагона должна быть от 1 до 64 мест. Введено: %', NEW."Количество мест";
TranLien$#     END IF;
TranLien$#     RETURN NEW;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_wagon_seats BEFORE INSERT OR UPDATE ON "Passengers"."Вагон"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_check_wagon_seats();
CREATE TRIGGER

```

Добавляем только вагоны поезда с количеством мест от 0 до 64.

```

TranLien=# INSERT INTO "Passengers"."Вагон" ("ID_вагона", "ID_типа вагона", "Номер вагона", "Количество мест")
TranLien=# VALUES (555, 1, 9, 150);
ERROR:  Ioeaea! Aianoeiinou aaaiia aie?ia auou io 1 ai 64 iano. Aaaaaai: 150
CONTEXT:  PL/pgSQL function "Passengers".fn_check_wagon_seats() line 4 at RAISE

```

```

TranLien=# INSERT INTO "Passengers"."Вагон" ("ID_вагона", "ID_типа вагона", "Номер вагона", "Количество мест")
TranLien=# VALUES (555, 1, 9, 60);
INSERT 0 1

```


f) ТРИГГЕР 6: Автоматическое переключение статуса места при покупке билета
(AFTER INSERT/UPDATE)

```
CREATE OR REPLACE FUNCTION "Passengers".fn_auto_update_seat_status()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW."ID_билета" > 0 THEN
        NEW."Статус" := 'Занято';
    ELSE
        NEW."Статус" := 'Свободно';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_seat_status BEFORE INSERT OR UPDATE ON
"Passengers"."Место"
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_auto_update_seat_status();
```

```
TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_auto_update_seat_status() RETURNS TRIGGER AS $$
TranLien$$ BEGIN
TranLien$$     IF NEW."ID_билета" > 0 THEN
TranLien$$         NEW."Статус" := 'Занято';
TranLien$$     ELSE
TranLien$$         NEW."Статус" := 'Свободно';
TranLien$$     END IF;
TranLien$$     RETURN NEW;
TranLien$$ END;
TranLien$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_seat_status BEFORE INSERT OR UPDATE ON "Passengers"."Место"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_auto_update_seat_status();
CREATE TRIGGER
```

```
TranLien=# INSERT INTO "Passengers"."Место" ("ID_места", "ID_билета", "ID_типа места", "Номер места", "Статус")
TranLien=# VALUES (88, 501, 2, 88, 'Свободно');
INSERT 0 1
```

```
TranLien=# SELECT * FROM "Passengers"."Место" WHERE "ID_места" = 88;
 ID_места | ID_билета | ID_типа места | Номер места | Статус
-----+-----+-----+-----+-----
      88 |      501 |           2 |          88 | Занято
(1 row)
```

Несмотря на то, что изначально место было указано как свободное, система обнаружила, что билет уже куплен, и автоматически изменила его статус на «занято».

ID_места	ID_билета	ID_типа места	Номер места	Статус
1	501	2	1	Занято
2	502	3	2	Занято
3	503	4	3	Занято
4	504	5	4	Занято
5	505	1	5	Занято
6	506	2	6	Занято
7	507	3	7	Занято
8	508	4	8	Занято
9	509	5	9	Занято
10	510	1	10	Занято
11	511	2	11	Занято
12	512	3	12	Занято
13	513	4	13	Занято
14	514	5	14	Занято
15	515	1	15	Занято
16	0	2	16	Свободно
17	0	3	17	Свободно
18	0	4	18	Свободно
19	0	5	19	Свободно

Это можно использовать для обновления статуса существующих билетов.

```
TranLien=# UPDATE "Passengers"."Место"
TranLien=# SET "ID_билета" = 516
TranLien=# WHERE "ID_места" = 16;
UPDATE 1
TranLien=# SELECT * FROM "Passengers"."Место" WHERE "ID_места" = 16;
```

ID_места	ID_билета	ID_типа места	Номер места	Статус
16	516	2	16	Занято

Когда обновляем ID_билета система поймет, что билет был куплен, и изменит его статус.

g) ТРИГГЕР 7: Контроль временных зон в маршрутах (BEFORE INSERT/UPDATE)

```
CREATE OR REPLACE FUNCTION "Passengers".fn_check_route_time() RETURNS
TRIGGER AS $$
BEGIN
    IF NEW."Время назначения" <= NEW."Время отправления" THEN
        RAISE EXCEPTION 'Ошибка: Время прибытия в назначение должно быть позже
времени отправления!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_route_time BEFORE INSERT OR UPDATE ON
"Passengers"."Маршрут"
FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_check_route_time();
```

```
TranLien=# CREATE OR REPLACE FUNCTION "Passengers".fn_check_route_time() RETURNS TRIGGER AS $$
TranLien$# BEGIN
TranLien$#     IF NEW."Время назначения" <= NEW."Время отправления" THEN
TranLien$#         RAISE EXCEPTION 'Ошибка: Время прибытия в назначение должно быть позже времени отправления!';
TranLien$#     END IF;
TranLien$#     RETURN NEW;
TranLien$# END;
TranLien$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
TranLien=# CREATE TRIGGER trg_route_time BEFORE INSERT OR UPDATE ON "Passengers"."Маршрут"
TranLien=# FOR EACH ROW EXECUTE PROCEDURE "Passengers".fn_check_route_time();
CREATE TRIGGER
```

```
TranLien=# INSERT INTO "Passengers"."Маршрут" ("ID_маршрута", "Название маршрута", "Точка отправления",
"Точка назначения", "Время отправления", "Время назначения")
TranLien=# VALUES (99, 'Ошибка-Тайм', 'СПб', 'Москва', '12:00:00-07', '10:00:00-07');
ERROR:  Ioεaea: A?aiy i?εauoey a iacia?aiea aie?ii auou iic?a a?aiaie ioi?aaεaiey!
CONTEXT:  PL/pgSQL function "Passengers".fn_check_route_time() line 4 at RAISE
```

Поскольку время отправления было позже времени назначения, это неверный маршрут, и он не будет добавлен.

Вывод

В ходе выполнения лабораторной работы были получены и закреплены практические навыки создания и применения процедурных объектов — хранимых процедур, триггерных функций и триггеров — в среде СУБД PostgreSQL с использованием консольного клиента SQL Shell (psql). Работа выполнялась на базе индивидуальной СУБД «Пассажирские перевозки» (Passengers)

- Реализация бизнес-логики через процедуры: Были разработаны и протестированы три хранимые процедуры (raise_suburban_prices, create_new_trip, get_daily_revenue). В ходе работы было освоено ключевое отличие процедур от функций: способность процедур напрямую управлять транзакциями (выполнять команду COMMIT внутри тела) и передавать агрегированные данные (например, суточную выручку) в вызывающий код через выходные параметры INOUT.
- Разработка триггеров уровня строки, срабатывающих до выполнения целевой операции (BEFORE INSERT/UPDATE), позволила внедрить жесткие правила валидации данных, которые невозможно реализовать стандартными ограничениями CHECK. На примере контроля вместимости вагонов, проверки статусов касс («Да»/«Нет») и временных интервалов маршрутов было доказано, что возвращение значения NULL или вызов исключения RAISE EXCEPTION триггером BEFORE эффективно блокирует запись некорректных данных в СУБД.
- Создание триггеров, срабатывающих после операций модификации данных (AFTER INSERT/UPDATE/DELETE), позволило успешно реализовать автономную систему журналирования. Применение встроенных контекстных переменных NEW, OLD и TG_OP позволило одной триггерной функции автоматически отслеживать тип операции и записывать хронологию изменений (регистрация билетов, изменение данных пассажиров) в изолированную таблицу логов (logs) без участия внешнего приложения.